SpaceX Falcon 9 First Stage Landing Prediction

Applied Data Science Capstone Project Report

- Student GitHub Repository: github.com/jatenor/spacex
- Deliverable: PDF Presentation Report & Machine Learning Pipeline
- Primary Objective: Predict First-Stage Reusability to Estimate Launch Cost Savings

1. Executive Summary (Rubric 1.3)

Project Goals & Business Context

Objective:

Predict if the SpaceX Falcon 9 first stage will land successfully to evaluate rocket reusability.

Business Impact:

SpaceX offers Falcon 9 launches for $62M vs. $165M+ for traditional non-reusable competitors. Prediction accuracy directly informs pricing models and mission insurance.

Key Finding:

Classification models achieved 83.33% accuracy on evaluation datasets, proving strong landing predictability.

End-to-End Methodology Pipeline

1. Data Collection:

Harvested launch data via SpaceX REST API & Web Scraping (Wikipedia).

2. Data Wrangling:

Handled null values, created binary target 'Class', and One-Hot encoded categorical variables.

3. Exploratory Analysis:

Used SQL, Seaborn charts, Folium maps, and Plotly Dash dashboards.

4. Machine Learning:

Trained Logistic Regression, SVM, Decision Tree, and k-NN with GridSearchCV.

2. Introduction & Problem Statement (Rubric 1.4)

Project Background

SpaceX Commercial Advantage:

By pioneering reusable first-stage boosters, SpaceX dramatically reduced the cost of access to space.

Cost Reduction Mechanics:

The first stage represents over 60% of the total launch cost. Recovering and refurbishing it allows multi-launch deployment with minimal incremental hardware expense.

Problem Statement

Core Question:

Can we predict whether a Falcon 9 booster stage will land successfully given payload mass, orbit type, launch site, and core history?

Target Variable:

Class = 1 (Successful Landing: RTLS / ASDS / Ocean)
Class = 0 (Unsuccessful / Uncontrolled Landing)

Value Proposition:

Empowers commercial competitors and payload operators to benchmark SpaceX launch risks and pricing structures.

3. Data Collection Methodologies (Rubric 1.5 & 1.6)

SpaceX REST API (Rubric 1.5)

API Endpoint:

https://api.spacexdata.com/v4/launches/past

Extraction Workflow:

- Filtered JSON payload specifically for Falcon 9 launches.
- Custom helper functions used:
  - getBoosterVersion()
  - getLaunchSite()
  - getPayloadData()
  - getCoreData()
- Compiled structured tabular dataset in Pandas.

GitHub Link: github.com/jatenor/spacex

Wikipedia Web Scraping (Rubric 1.6)

Scraping Target:

List of Falcon 9 and Falcon Heavy launches (Wikipedia)

Scraping Workflow:

- Executed HTTP GET requests to fetch HTML document.
- Parsed HTML structure using BeautifulSoup.
- Extracted table rows: Flight No, Launch Site, Payload Mass, Orbit, Customer, Landing Outcome.
- Executed regular expression string cleaning.

GitHub Link: https://github.com/joaoatenor/spacex/blob/main/jupyter-labs-webscraping.ipynb

4. Data Wrangling Methodology (Rubric 1.7)

Data Cleaning & Feature Engineering

1. Handling Missing Data:

Identified null entries in numeric columns (e.g., PayloadMass). Imputed missing payload values using the mean payload mass of existing records.

2. Binary Classification Label Creation (Class):

Converted complex landing outcome strings into a binary target column:
- Class = 1: Successful recovery ('True Ocean', 'True RTLS', 'True ASDS')
- Class = 0: Unsuccessful / Failed recovery ('False Ocean', 'False RTLS', 'False ASDS', 'None')

3. Feature Transformation (One-Hot Encoding):

Applied pd.get_dummies() to categorical features (LaunchSite, Orbit, BoosterVersion) to create a numerical feature matrix suitable for machine learning algorithms.

GitHub Repository Link: github.com/jatenor/spacex

5. Exploratory Data Analysis: Visuals & SQL (Rubric 1.8, 1.9, 1.11, 1.12)

EDA Data Visualizations (Rubric 1.8 & 1.11)

• Flight No. vs. Payload Mass:

Early flights (<20) carried lower payload masses and had higher failure rates; later heavy payload missions achieved high success.

• Launch Site Success Rates:

KSC LC-39A and VAFB SLC-4E achieved higher overall landing success rates compared to early CCAFS SLC-40 tests.

• Orbit Performance:

High-altitude orbits (ES-L1, GEO, HEO, SSO) show near 100% success; SO orbit has the lowest.

• Annual Trend:

Clear upward trajectory in success rate from 2010 to 2020.

EDA with SQL Queries (Rubric 1.9 & 1.12)

Key Queries Performed:

• Launch Sites Extraction:
SELECT DISTINCT LaunchSite FROM SPACEX;
• Total Payload Delivered:
SELECT SUM(PayloadMass) FROM SPACEX WHERE ...
• First Successful Drone Ship Landing:
SELECT MIN(Date) FROM SPACEX WHERE LandingOutcome = ...
• Yearly Success Frequency Analysis:
SELECT SUBSTR(Date, 1, 4), COUNT(*) ... GROUP BY ...

GitHub Repository Link: github.com/jatenor/spacex

]

6. Interactive Visual Analytics (Rubric 1.10, 1.13, 1.14)

Geospatial Maps (Folium - Rubric 1.13)

Interactive Map Features:

• Placed color-coded markers (Green = Success, Red = Failure) for each launch site.
• Constructed MarkerClusters to manage launch density.
• Computed Haversine proximity distances from pads to coastline, railways, highways, and cities.

Geospatial Insight:

Launch pads are strictly situated along coastlines to ensure safety during abort trajectories and debris falling over ocean waters.

Plotly Dash Dashboard (Rubric 1.14)

Interactive Application Features:

• Site Selection Dropdown: Renders dynamic pie charts comparing success vs. failure ratios per site or across all sites.
• Payload Range Slider: Dynamically filters payload mass scatter plots ($0 - 10,000 kg$).

Dashboard Insight:

Booster version FT consistently demonstrated superior recovery performance across all payload weight classes.

7. Predictive Modeling & ML Results (Rubric 1.15)

Model Training, Hyperparameter Tuning & Evaluation

Classification Pipeline Setup:

- Split data into 80% Training and 20% Test sets.
- Standardized numerical features using StandardScaler.
- Applied GridSearchCV with 10-fold cross-validation across 4 algorithms:

Model Performance Evaluation:

1. Logistic Regression (C, penalty): Training Score = 84.72% | Test Score = 83.33%
2. Support Vector Machine (C, kernel): Training Score = 84.72% | Test Score = 83.33%
3. Decision Tree (criterion, max_depth): Training Score = 88.89% | Test Score = 83.33%
4. k-Nearest Neighbors (n_neighbors, p): Training Score = 84.72% | Test Score = 83.33%

Best Model Selection & Conclusion:

Decision Tree Classifier is selected as the top model due to its high training accuracy (88.89%) and transparent decision rule tree for engineering interpretability.